REAL-TIME AI-DRIVEN AGILE REQUIREMENT HARMONIZATION

R26-SE-039

Research Project Checklist

A.G Jinad Induvidwa – IT22251114

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

Contents

REAL-TIME AI-DRIVEN AGILE REQUIREMENT 1
INTRODUCTION 3
METHODOLOGY 4
SYSTEM ARCHITECTURE 5
EXPECTED OUTCOMES 6
REPO LINKS (GITHUB) 7

INTRODUCTION

Agile software development relies heavily on continuous communication among stakeholders to gather, refine, and validate software requirements. However, modern agile meetings often involve multiple participants, fast-paced discussions, overlapping conversations, and evolving requirements, which can lead to ambiguity, inconsistencies, and logical contradictions. These challenges become more critical in distributed or hybrid development environments where communication gaps may negatively impact software quality, project timelines, and downstream Quality Assurance (QA) workflows. Traditional requirement engineering approaches primarily depend on manual note-taking, post-meeting documentation, and human interpretation, making the process time-consuming, error-prone, and difficult to scale in real-time agile environments.

To address these limitations, this research proposes the design, development, and evaluation of a real-time AI-driven Agile Requirement Harmonization Pipeline capable of automatically processing live agile meetings and transforming spoken discussions into structured, validated software requirements. The proposed system integrates advanced Automatic Speech Recognition (ASR), speaker diarization, semantic intelligence, and Retrieval-Augmented Generation (RAG) technologies to create an intelligent end-to-end requirement engineering framework. By leveraging Faster-Whisper and Pyannote Audio, the system aims to accurately transcribe and identify multiple speakers in real-time while maintaining a Word Error Rate (WER) below 15% for technical software engineering terminology and achieving a Real-Time Factor (RTF) of less than 1.0.

Beyond transcription, the research introduces a semantic conflict detection mechanism powered by Llama 3 and ChromaDB to continuously analyze meeting conversations and identify logical contradictions or requirement mismatches within a strict 5-second processing window. When conflicts are detected, the system immediately activates a visual “Red Warning” protocol through a web-based dashboard, enabling stakeholders to rapidly identify and resolve inconsistencies during the meeting itself. Additionally, the proposed framework incorporates paralinguistic risk assessment by analyzing voice cues such as hesitation duration and jitter levels to estimate requirement ambiguity and calculate an “Acoustic Risk Score” for spoken statements.

Furthermore, the system automatically generates structured JSON-based user stories using RAG and prompt chaining techniques while evaluating their quality using INVEST-based validation metrics before storage. By automating transcription, conflict detection, requirement validation, and user story generation

METHODOLOGY

This project adopts a Design Science Research (DSR) approach, as it focuses on the creation, implementation, and evaluation of a novel technological artifact [18].

- **Streaming Diarization and ASR**: Raw audio captured via WebRTC is piped through WebSockets to an acoustic layer that performs real-time transcription and diarization concurrently.

- **Paralinguistic Feature Extraction**: Algorithms extract acoustic markers (e.g., jitter, silence duration) to calculate an "Acoustic Risk Score" .

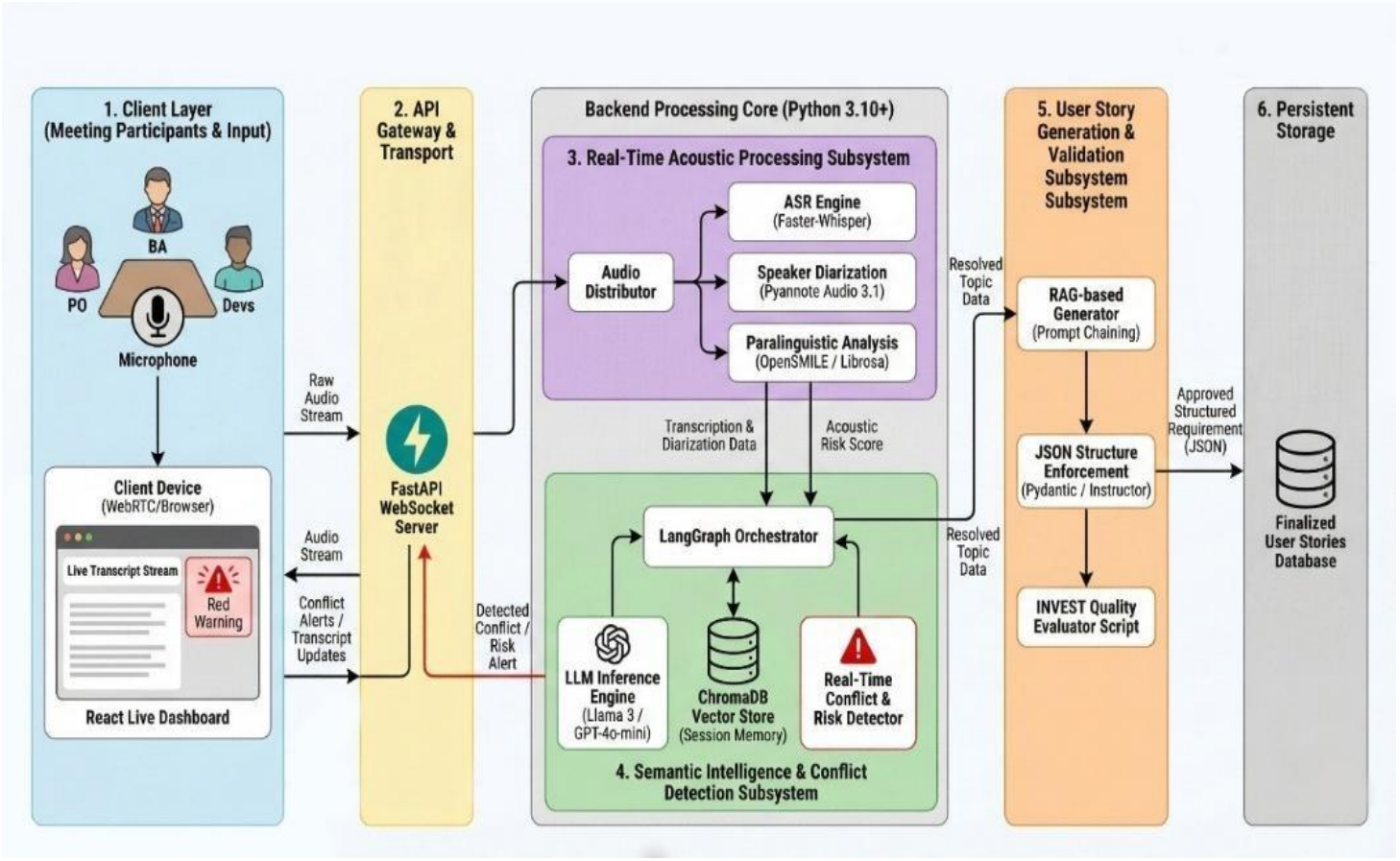
- **Sliding Window Semantic Analysis:** As transcripts are generated, an LLM crossreferences new segments against a vector-based session memory (ChromaDB) to identify logical contradictions in near real-time .

- **RAG and Prompt Chaining:** Upon conflict resolution, the system synthesizes corrected transcript data using RAG to output structured JSON requirements/user stories .

- **FastAPI & React**: FastAPI manages asynchronous WebSocket streams for low-latency transport, while React powers the live dashboard UI .

SYSTEM ARCHITECTURE

Figure 1: System Architecture



EXPECTED OUTCOMES

Acoustic Processing and Diarization

The expected outcome of this component is the development of a highly accurate real-time speech processing pipeline capable of transcribing agile meetings and identifying multiple speakers simultaneously. The system is expected to achieve a Word Error Rate (WER) below 15% for technical software engineering terminology while maintaining a Real-Time Factor (RTF) below 1.0. This outcome ensures reliable meeting transcription, clear speaker separation, and improved traceability of stakeholder discussions for downstream processing tasks.

Paralinguistic Risk Assessment

This component is expected to produce an intelligent risk analysis mechanism capable of detecting requirement uncertainty and ambiguity through voice-based paralinguistic features such as hesitation duration, speech pauses, and vocal jitter. The final outcome will be the generation of an "Acoustic Risk Score" that continuously evaluates the confidence level of spoken requirements during meetings, enabling teams to identify potentially unclear or risky requirements at an early stage.

Real-Time Semantic Conflict Detection

The expected outcome of this component is the implementation of a semantic intelligence framework that can analyze live meeting transcripts and detect logical contradictions or requirement mismatches in real-time. By utilizing Llama 3 and ChromaDB, the system is expected to provide "Warning/No Warning" decisions within a maximum latency of 5 seconds, improving requirement consistency and reducing the likelihood of conflicting stakeholder expectations.

Immediate Visual Feedback (Red Warning Protocol)

This component is expected to deliver a responsive web-based dashboard capable of providing immediate visual alerts whenever semantic conflicts or platform inconsistencies are detected. The "Red Warning" mechanism will support rapid stakeholder awareness and decision-making during agile meetings, helping teams resolve issues instantly rather than discovering them during later development or testing phases.

Automated User Story Generation and Quality Scoring

The expected outcome of this component is the creation of an automated requirement synthesis module capable of converting resolved meeting discussions into structured JSON user stories. The system will also evaluate generated stories against INVEST criteria and assign a measurable Testability Score before database storage. This outcome is expected to reduce manual documentation effort, improve requirement quality, and provide a standardized input for downstream QA and software testing workflows.

REPO LINKS (GITHUB)

1. Frontend Application
   Frontend Repository
2. Intelligent User Story Generator Service
   Intelligent User Story Generator Service
3. Intelligent Voice Parser Service
   Intelligent Voice Parser Service
4. Authentication Service
   Authentication Service

The proposed AI-driven Agile Requirement Harmonization platform is developed using a microservices-based architecture to ensure scalability, modularity, maintainability, and independent deployment of system components. Each major research functionality is implemented as an individual service that communicates through APIs, enabling efficient processing and real-time integration across the platform. The overall system consists of multiple interconnected repositories, including the frontend application, intelligent voice parser service, intelligent user story generator service, and authentication service.